ТЕХНИЧЕСКОЕ РУКОВОДСТВО

Selfvoy: оплата внутри приложения и техническая публикация

Подписка через App Store и Google Play, проверка покупок на сервере, изменения в базе и протоколе, сборка и выгрузка релизов из CI. Привязано к коду репозитория: C++-сервер, Python AI-сервис, клиент Capacitor (web/), нативные клиенты.

Главные решения

- Подписка Selfvoy Pro — цифровая услуга, поэтому внутри приложения её можно продавать **только** через Apple In-App Purchase и Google Play Billing. Свой эквайринг для неё запрещён (кроме исключений по странам, раздел 1).
- Источник правды о подписке — **ваш сервер**, а не телефон. Клиент только запускает покупку и передаёт её на проверку.
- C++-сервер не умеет исходящий HTTPS, поэтому проверку у Apple и Google и приём их уведомлений делает **Python AI-сервис** (новый модуль billing). Сервер вызывает его по внутреннему токену, как уже делает для ИИ.
- Android: **Play Billing Library 8+** (обязательно с 31.08.2026). iOS: **StoreKit 2**. Сборка iOS — **Xcode 26**.
- Быстрый путь — RevenueCat вместо собственной проверки (раздел 4). Он экономит 1–2 недели, но добавляет внешний сервис и комиссию после 2 500 USD выручки в месяц.

Содержание

	Раздел
1	Что можно и что нельзя продавать через приложение, комиссии, Россия
2	Модель подписки: продукты, ID, что даёт Pro
3	Архитектура: кто кому что передаёт
4	Свой сервер проверки или RevenueCat
5	База данных: миграция
6	Протокол: новые методы и события
7	Клиент iOS: StoreKit 2
8	Клиент Android: Play Billing Library 8
9	Сервер: проверка покупок и уведомления Apple/Google
10	Настройка App Store Connect и Play Console под покупки
11	Экран оплаты: обязательные элементы
12	Тестирование покупок
13	Техническая публикация: подпись, сборка, CI/CD
14	Прод-инфраструктура
15	План работ и источники

1. Что продавать через магазины, а что нельзя

Что продаёте	Как принимать оплату	Правило
Подписка Selfvoy Pro (ИИ-посланник, лимиты, функции)	**Только** Apple IAP / Google Play Billing	Apple 3.1.1; политика платежей Google Play
Бронь столика, депозит, оплата реальной услуги или товара	Обычный эквайринг (ЮKassa, CloudPayments, Stripe и т. п.), **не** IAP	Apple 3.1.3(e)/3.1.5; у Google — физические товары и услуги вне Play Billing
Подписка, купленная на сайте или на другой платформе	Доступ по аккаунту разрешён	Apple 3.1.3(b) «мультиплатформенные сервисы», если та же подписка продаётся и через IAP
Ссылка из приложения на оплату на сайте	США: разрешено обоими магазинами (Google — программа external content links, сборы с 01.10.2026). ЕС: новые условия Apple с 01.10.2026. Остальные страны — нельзя	Сверьте с консолью: условия меняются по решениям судов

Комиссии (на 26.10.2026)

- **Apple:** 15% для участников App Store Small Business Program (выручка до 1 млн USD в год — подавайте заявку сразу) и для подписок после первого года; иначе 30%. В ЕС с 01.10.2026 отдельные ставки (26%/15% для IAP).
- **Google:** базово 15% для подписок. С 30.06.2026 в США, Великобритании и ЕЭЗ новая модель сборов (в том числе для альтернативной оплаты) — точные ставки смотрите в Play Console для вашего аккаунта.
- Налоги (НДС и аналоги) магазины удерживают и платят сами в большинстве стран; вам приходит выручка за вычетом комиссии.

Пользователи из России

- Google приостановил платёжную систему Google Play для пользователей в России в 2022 году; Apple не принимает российские карты. Пользователи из РФ, как правило, не смогут оплатить подписку через эти магазины.
- Для Android-аудитории в России — **RuStore Pay SDK** (карты, СБП). Старый RuStore BillingClient SDK перестал обрабатывать платежи с 01.08.2026 — только Pay SDK.
- На iOS в России законного обходного пути для цифровой подписки внутри приложения нет: продажа на сайте + доступ по аккаунту возможна, но **ссылаться** на неё из iOS-приложения в РФ нельзя. Это бизнес-решение — обсудите с юристом.
- Выплаты: Google отключил выплаты на российские счета (12.2024); для приёма денег от магазинов нужен зарубежный банковский счёт или юрлицо.

2. Модель подписки

	Free	Pro
Переговоры посланников	5 в месяц	Без лимита
Голосовой ввод	30 минут в месяц	Без лимита
Модель ИИ	Базовая	Сильнее + длинная память
Интеграции (календарь, почта)	1	Все

Лимиты — пример; главное, чтобы проверка шла на сервере по «праву доступа» (entitlement) `pro`, а не по ID продукта.

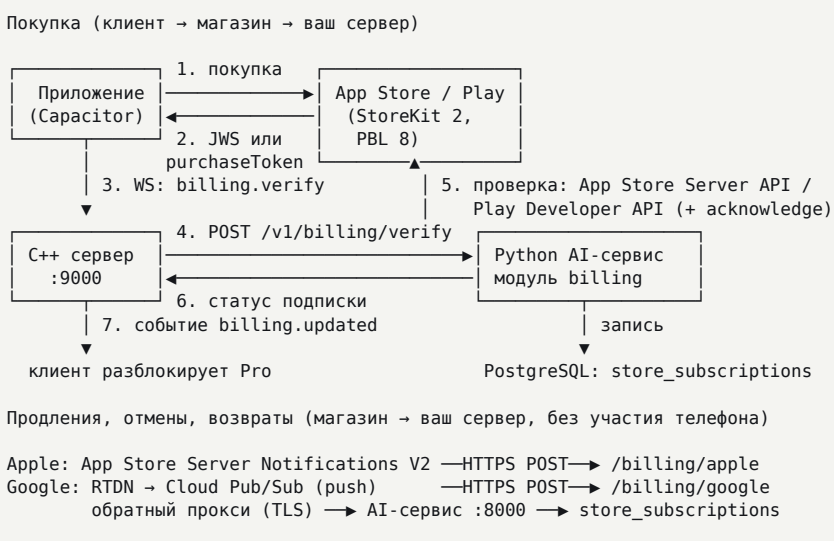
Идентификаторы продуктов

	App Store	Google Play
Группа / продукт	Группа подписок «Selfvoy Pro»	Подписка `selfvoy_pro`
Месяц	Продукт `selfvoy_pro_monthly`	Базовый план `monthly`
Год	Продукт `selfvoy_pro_yearly` (уровень тот же)	Базовый план `yearly`
Пробный период	Вводное предложение: 7 дней бесплатно	Предложение `trial-7d` на базовом плане
Привязка к аккаунту	`appAccountToken` = UUID пользователя	`obfuscatedAccountId` = тот же UUID

ID продуктов после создания не меняются и не используются повторно даже после удаления — выбирайте сразу.

3. Архитектура

Схема потоков



- **Почему через AI-сервис:** в C++-сервере нет HTTPS-клиента (это же мешает Google-интеграциям), а в Python есть `httpx` и официальная библиотека Apple. Сервис уже закрыт токеном `AURA_AI_TOKEN` и не опубликован наружу.
- **Наружу публикуются только два пути** `/billing/apple` и `/billing/google` через обратный прокси; остальной AI-сервис по-прежнему недоступен из интернета. Подлинность уведомлений проверяется подписью (Apple) и OIDC-токеном Pub/Sub (Google).
- **Право доступа читает C++-сервер** из таблицы подписок при входе и перед действием с лимитом. Кэш — не дольше минуты, чтобы возврат денег отключал Pro быстро.
- **Повторы безопасны:** магазины шлют одно уведомление несколько раз; ID уведомления хранится с уникальным индексом, повтор — no-op.

4. Свой сервер проверки или RevenueCat

	Свой (рекомендуется для Selfvoy)	RevenueCat
Клиент	Свой небольшой Capacitor-плагин (Swift + Kotlin, ~400 строк) или нативный код	Готовый плагин `@revenuecat/purchases-capacitor`
Сервер	Модуль billing в AI-сервисе: проверка + 2 вебхука	Один вебхук RevenueCat + REST API
Срок	≈ 2–3 недели	≈ 1 неделя
Цена	Бесплатно	Бесплатно до 2 500 USD выручки в месяц, дальше 1%
Данные	Остаются у вас	Покупки и ID пользователей у третьей стороны — указать в политике и App Privacy
Россия (RuStore)	Добавляется третьим магазином в ту же схему	RuStore не поддерживается — отдельная интеграция всё равно нужна

Рекомендация: свой вариант. Вам всё равно нужен RuStore, сервер уже является источником правды о пользователе, а в проекте строгие правила приватности. Ниже описан именно он; если выберете RevenueCat — разделы 7–9 заменяются его SDK и одним обработчиком вебхука, разделы 5, 6, 10–13 остаются.

5. База данных: миграция

Файл `schema/migrations/2026_11_billing.sql`; те же таблицы — в `schema/schema.sql` и в `docs/DATABASE.md`. Встроенная БД сервера (database_embedded) тоже должна их понимать.

schema/migrations/2026_11_billing.sql

```
-- Постоянный UUID для привязки покупок к аккаунту (appAccountToken / obfuscatedAccountId)
ALTER TABLE users ADD COLUMN IF NOT EXISTS billing_uuid UUID NOT NULL DEFAULT gen_random_uuid();
CREATE UNIQUE INDEX IF NOT EXISTS users_billing_uuid_idx ON users (billing_uuid);

CREATE TABLE IF NOT EXISTS store_subscriptions (
    id              BIGSERIAL    PRIMARY KEY,
    user_id         BIGINT       NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    store           TEXT         NOT NULL CHECK (store IN ('apple', 'google', 'rustore')),
    product_id      TEXT         NOT NULL,          -- selfvoy_pro_monthly | selfvoy_pro:monthly
    store_key       TEXT         NOT NULL,          -- Apple originalTransactionId | Google purchaseToken
    status          TEXT         NOT NULL CHECK (status IN
                      ('active', 'grace', 'on_hold', 'paused', 'canceled', 'expired', 'revoked')),
    expires_at      TIMESTAMPTZ,
    will_renew      BOOLEAN      NOT NULL DEFAULT TRUE,
    is_trial        BOOLEAN      NOT NULL DEFAULT FALSE,
    environment     TEXT         NOT NULL DEFAULT 'production',   -- production | sandbox
    updated_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    UNIQUE (store, store_key)
);
CREATE INDEX IF NOT EXISTS store_subscriptions_user_idx ON store_subscriptions (user_id);

CREATE TABLE IF NOT EXISTS billing_events (
    id              BIGSERIAL    PRIMARY KEY,
    store           TEXT         NOT NULL,
    event_id        TEXT         NOT NULL,          -- notificationUUID | Pub/Sub messageId
    kind            TEXT         NOT NULL,          -- DID_RENEW, SUBSCRIPTION_RENEWED, ...
    store_key       TEXT,
    received_at     TIMESTAMPTZ  NOT NULL DEFAULT now(),
    processed_at    TIMESTAMPTZ,
    UNIQUE (store, event_id)                        -- повтор уведомления = no-op
);

-- AI-сервис пишет только сюда (роль aura_ai, deploy/postgres/20-app-roles.sh)
GRANT SELECT (id, billing_uuid) ON users TO aura_ai;
GRANT SELECT, INSERT, UPDATE ON store_subscriptions, billing_events TO aura_ai;
GRANT USAGE, SELECT ON SEQUENCE store_subscriptions_id_seq, billing_events_id_seq TO aura_ai;
```

- **Право доступа:** Pro есть, если у пользователя хотя бы одна запись со статусом `active`, `grace` или `canceled` (отменено автопродление, но оплаченный срок не кончился) и `expires_at > now()`.
- **Удаление аккаунта** удаляет записи каскадом, но **не отменяет** подписку в магазине: перед удалением покажите предупреждение и ссылку «Управлять подпиской».
- **Сырые чеки не храним** — только нужные поля. Подписанный JWS/токен можно получить снова из API магазина.

6. Протокол: новые методы и события

По правилу проекта изменение протокола — одним коммитом: сервер, все клиенты, `docs/PROTOCOL.md`, чекеры `tools/check_*_protocol.py` и сценарий `tools/e2e.py`.

Тип	payload запроса	Ответ / событие
`billing.status`	—	`{plan:'free'|'pro', store, product_id, expires_at, will_renew, is_trial, billing_uuid, limits:{negotiations:{used,max,resets_at}, voice_min:{...}}}`

Тип	payload запроса	Ответ / событие
`billing.verify`	`{store:'apple', jws}` или `{store:'google', purchase_token, product_id}` или `{store:'rustore', purchase_id}`	То же, что billing.status. Ошибки: `billing_invalid` (подпись/пакет/продукт не наши), `billing_conflict` (покупка привязана к другому аккаунту), `billing_unavailable` (магазин не ответил — клиент повторит)
событие `billing.updated`	—	Сервер шлёт всем соединениям пользователя при смене статуса (покупка, продление, возврат, истечение)
ошибка `plan_limit`	—	Любой метод с лимитом: `{limit:'negotiations', used, max, resets_at}` — клиент показывает экран Pro

Пример обмена

```
→ {"id":41, "type":"billing.verify",
   "payload":{"store":"google","purchase_token":"opkd…","product_id":"selfvoy_pro"}}
← {"id":41, "type":"ok", "payload":{"plan":"pro","store":"google","product_id":"selfvoy_pro:monthly",
   "expires_at":"2026-11-26T09:12:00Z","will_renew":true,"is_trial":false, ...}}
← {"type":"event", "event":"billing.updated", "payload":{"plan":"pro", ...}}
```

Внутренний HTTP AI-сервиса (заголовок `X-Aura-Token`, только из C++-сервера):

Метод	Назначение
`POST /v1/billing/verify`	{user_id, billing_uuid, store, jws | purchase_token+product_id | purchase_id} → проверка, запись в store_subscriptions, нормализованный статус
`POST /billing/apple`	Публичный (через прокси): App Store Server Notifications V2, тело {signedPayload}
`POST /billing/google`	Публичный (через прокси): Pub/Sub push, заголовок Authorization: Bearer <OIDC JWT>
`POST /billing/rustore`	Публичный: уведомления RuStore (по их документации, с проверкой подписи)

Как AI-сервис сообщает C++-серверу об изменении из вебхука: проще всего — сервер раз в 30–60 с проверяет `store_subscriptions.updated_at` для онлайн-пользователей и рассылает `billing.updated`; клиенты дополнительно вызывают `billing.status` при возврате приложения на экран.

7. Клиент iOS: StoreKit 2

Для клиента Capacitor — небольшой плагин `SelfvoyBilling` (Swift в `web/ios/App`, Kotlin в `web/android`) с методами `products`, `purchase`, `restore`, `manage` и событием `transaction`. Для нативного SwiftUI-клиента — тот же код в сервисе рядом с `AppStore.swift`.

Billing.swift

```
import StoreKit

final class Billing {
    static let ids = ["selfvoy_pro_monthly", "selfvoy_pro_yearly"]
    private var updates: Task<Void, Never>?

    // Запустить при старте приложения: продления, Ask to Buy, покупки с другого устройства.
    func start(send: @escaping (String) async throws -> Void) {
        updates = Task.detached {
            for await result in Transaction.updates {
                guard case .verified(let tx) = result else { continue }
                do { try await send(result.jwsRepresentation); await tx.finish() } catch { }
            }
        }
    }

    func products() async throws -> [Product] { try await Product.products(for: Self.ids) }

    // billingUUID приходит с сервера (billing.status) — привязывает покупку к аккаунту.
    func buy(_ product: Product, billingUUID: UUID,
             send: (String) async throws -> Void) async throws -> Bool {
        let result = try await product.purchase(options: [.appAccountToken(billingUUID)])
        switch result {
        case .success(let verification):
            guard case .verified(let tx) = verification else { return false }
            try await send(verification.jwsRepresentation)   // WS: billing.verify {store:"apple", jws}
            await tx.finish()                                // только после ответа сервера
            return true
        case .pending:       return false   // ждём Transaction.updates
        case .userCancelled: return false
        @unknown default:    return false
        }
    }

    func restore() async throws { try await AppStore.sync() }   // кнопка «Восстановить покупки»
}

// «Управлять подпиской»:  try await AppStore.showManageSubscriptions(in: windowScene)
```

- Не вызывайте `finish()` до ответа сервера: незавершённая транзакция придёт снова в `Transaction.updates`, и покупка не потеряется при обрыве сети.
- Цены показывайте только из `product.displayPrice` — валюта и налоги зависят от страны.
- В Xcode: Signing & Capabilities → **In-App Purchase**. Для локальных тестов — файл `Selfvoy.storekit` (StoreKit Configuration), подключить в схеме Run.
- Пробный период показывайте, только если `await product.subscription?.isEligibleForIntroOffer` = true.

8. Клиент Android: Play Billing Library 8

app/build.gradle.kts

```
dependencies { implementation("com.android.billingclient:billing-ktx:8.0.0") }  // 8.x или новее
```

Billing.kt

```
class Billing(private val context: Context, private val send: suspend (String, String) -> Unit) {
    private val scope = CoroutineScope(Dispatchers.Main + SupervisorJob())

    private val client = BillingClient.newBuilder(context)
        .setListener { result, purchases ->
            if (result.responseCode == BillingClient.BillingResponseCode.OK) purchases?.forEach(::handle)
        }
        .enablePendingPurchases(PendingPurchasesParams.newBuilder().enableOneTimeProducts().build())
        .enableAutoServiceReconnection()                        // PBL 8
        .build()

    fun connect(onReady: () -> Unit) = client.startConnection(object : BillingClientStateListener {
        override fun onBillingSetupFinished(r: BillingResult) { if (r.responseCode == 0) onReady() }
        override fun onBillingServiceDisconnected() {}
    })

    fun buy(activity: Activity, details: ProductDetails, basePlanId: String, billingUuid: String) {
        val offer = details.subscriptionOfferDetails!!.first { it.basePlanId == basePlanId }
        val params = BillingFlowParams.newBuilder()
            .setProductDetailsParamsList(listOf(
                BillingFlowParams.ProductDetailsParams.newBuilder()
                    .setProductDetails(details).setOfferToken(offer.offerToken).build()))
            .setObfuscatedAccountId(billingUuid)                // тот же UUID, что у Apple
            .build()
        client.launchBillingFlow(activity, params)
    }

    // Подтверждение (acknowledge) делает СЕРВЕР; без него через 3 дня Google вернёт деньги.
    private fun handle(p: Purchase) {
        if (p.purchaseState != Purchase.PurchaseState.PURCHASED || p.isAcknowledged) return
        scope.launch { send(p.purchaseToken, p.products.first()) }   // WS: billing.verify
    }

    // При каждом открытии приложения — подобрать покупки, сделанные без нас.
    fun resync() = client.queryPurchasesAsync(
        QueryPurchasesParams.newBuilder().setProductType(BillingClient.ProductType.SUBS).build()
    ) { _, list -> list.forEach(::handle) }
}
```

- Список продуктов — `queryProductDetailsAsync` с `ProductType.SUBS` и ID `selfvoy_pro`; цену берите из `pricingPhases` выбранного предложения.
- «Управлять подпиской» — ссылка `https://play.google.com/store/account/subscriptions?sku=selfvoy_pro&package=<applicationId>`.
- Нативный клиент `android/app`: тот же код; не забудьте `targetSdk 36`. Для RuStore — отдельный класс на Pay SDK с тем же интерфейсом, выбор по магазину установки.

9. Сервер: проверка покупок и уведомления

Новый модуль `ai/aura_ai/billing.py` и тесты `ai/tests/test_billing.py`. Зависимости: `app-store-server-library` (официальная библиотека Apple), `google-auth` (сервисный аккаунт и проверка OIDC-токена Pub/Sub), `httpx` (уже есть).

9.1. Apple

Проверка Apple

```
from appstoreserverlibrary.models.Environment import Environment
from appstoreserverlibrary.signed_data_verifier import SignedDataVerifier

# Корневые сертификаты Apple (AppleRootCA-G3.cer и др.) — файлами в образе, не из сети при старте.
verifier = SignedDataVerifier(root_certs, enable_online_checks=True,
                              environment=Environment.PRODUCTION,     # для TestFlight — SANDBOX
                              bundle_id=cfg.apple_bundle_id, app_apple_id=cfg.apple_app_id)

def verify_apple(user, jws: str) -> dict:
    tx = verifier.verify_and_decode_signed_transaction(jws)   # подпись, bundle id, окружение
    if tx.productId not in APPLE_PRODUCTS:
        raise BillingInvalid("unknown product")
    if tx.appAccountToken and str(tx.appAccountToken) != str(user.billing_uuid):
        raise BillingConflict()                                 # куплено другим аккаунтом
    status = "revoked" if tx.revocationDate else ("active" if tx.expiresDate > now_ms() else "expired")
    return upsert(user.id, "apple", tx.productId, key=tx.originalTransactionId,
                  status=status, expires_at=tx.expiresDate, env=tx.environment)

def on_apple_notification(body: dict) -> None:                 # POST /billing/apple
    n = verifier.verify_and_decode_notification(body["signedPayload"])
    if not remember_event("apple", n.notificationUUID, n.notificationType):
        return                                                  # повтор — уже обработали
    tx = verifier.verify_and_decode_signed_transaction(n.data.signedTransactionInfo)
    renewal = verifier.verify_and_decode_renewal_info(n.data.signedRenewalInfo)
    apply_apple(n.notificationType, n.subtype, tx, renewal)    # таблица ниже
```

Для запросов к Apple (история, статус по `originalTransactionId`) — `AppStoreServerAPIClient` с ключом In-App Purchase (.p8), `key_id`, `issuer_id`. На уведомление отвечайте 200 быстро; тяжёлую работу — после ответа.

9.2. Google

Проверка Google

```
API = "https://androidpublisher.googleapis.com/androidpublisher/v3/applications"

def verify_google(user, token: str, product_id: str) -> dict:
    r = http.get(f"{API}/{cfg.android_package}/purchases/subscriptionsv2/tokens/{token}",
                 headers=auth_headers())          # сервисный аккаунт, scope androidpublisher
    sub = r.json()
    owner = sub.get("externalAccountIdentifiers", {}).get("obfuscatedExternalAccountId")
    if owner != str(user.billing_uuid):
        raise BillingConflict()
    item = sub["lineItems"][0]
    if item["productId"] != "selfvoy_pro":
        raise BillingInvalid("unknown product")
    if sub.get("acknowledgementState") == "ACKNOWLEDGEMENT_STATE_PENDING":
        http.post(f"{API}/{cfg.android_package}/purchases/subscriptions/"
                  f"{item['productId']}/tokens/{token}:acknowledge", headers=auth_headers())
    if sub.get("linkedPurchaseToken"):                  # смена плана: старый токен заменён новым
        mark_replaced("google", sub["linkedPurchaseToken"])
    return upsert(user.id, "google", f"selfvoy_pro:{item['offerDetails']['basePlanId']}",
                  key=token,
                  status=GOOGLE_STATE[sub["subscriptionState"]], expires_at=item["expiryTime"],
                  will_renew=item.get("autoRenewingPlan", {}).get("autoRenewEnabled", False))

def on_google_push(headers, body) -> None:                      # POST /billing/google
    verify_pubsub_oidc(headers["Authorization"], audience=cfg.pubsub_audience)
    msg = body["message"]
    if not remember_event("google", msg["messageId"], "rtdn"):
        return
    data = json.loads(base64.b64decode(msg["data"]))
    if "subscriptionNotification" in data:                      # тип не важен — берём актуальное
        refresh_google(data["subscriptionNotification"]["purchaseToken"])   # subscriptionsv2.get
```

Методы v1 `subscriptions.get/revoke/cancel` устарели — используйте `subscriptionsv2`. Поле `subscriptionId` в RTDN устарело без замены: продукт берите из ответа subscriptionsv2.get. При смене плана старый токен приходит в `linkedPurchaseToken` — помечайте старую запись заменённой.

9.3. Как статусы магазинов переводятся в наши

Наш статус	Apple (notificationType / состояние)	Google (subscriptionState)	Pro?
`active`	SUBSCRIBED, DID_RENEW, DID_CHANGE_RENEWAL_STATUS (AUTO_RENEW_ENABLED)	SUBSCRIPTION_STATE_ACTIVE	да
`canceled`	DID_CHANGE_RENEWAL_STATUS (AUTO_RENEW_DISABLED)	SUBSCRIPTION_STATE_CANCELED	до expires_at
`grace`	DID_FAIL_TO_RENEW + subtype GRACE_PERIOD	SUBSCRIPTION_STATE_IN_GRACE_PERIOD	да
`on_hold`	DID_FAIL_TO_RENEW без grace (billing retry)	SUBSCRIPTION_STATE_ON_HOLD	нет
`paused`	—	SUBSCRIPTION_STATE_PAUSED	нет
`expired`	EXPIRED, GRACE_PERIOD_EXPIRED	SUBSCRIPTION_STATE_EXPIRED	нет
`revoked`	REFUND, REVOKE	SUBSCRIPTION_STATE_REVOKED (возврат / отзыв)	нет, сразу

9.4. Секреты (в `.env`, шаблон — `.env.example`)

Переменная	Что это
`AURA_APPLE_BUNDLE_ID`, `AURA_APPLE_APP_ID`	Bundle ID и числовой Apple ID приложения
`AURA_APPLE_IAP_KEY_ID`, `AURA_APPLE_ISSUER_ID`, `AURA_APPLE_IAP_KEY`	Ключ In-App Purchase (.p8) для App Store Server API
`AURA_APPLE_ENV`	`production` | `sandbox`
`AURA_ANDROID_PACKAGE`	applicationId
`AURA_GOOGLE_SA_JSON`	JSON сервисного аккаунта с доступом к Play Developer API
`AURA_PUBSUB_AUDIENCE`, `AURA_PUBSUB_SA_EMAIL`	Ожидаемые audience и отправитель OIDC-токена push-подписки
`AURA_RUSTORE_*`	Ключи RuStore Pay API (если подключаете RuStore)

Тесты без сети: подписанные тестовые JWS (библиотека Apple умеет работать с тестовыми сертификатами), моки ответов Google через `httpx.MockTransport`, проверка идемпотентности и конфликта аккаунтов.

10. Настройка магазинов под покупки

App Store Connect

1. Business → **Paid Apps Agreement**: принять, заполнить налоги и банковский счёт. Пока статус не «Active», покупки не работают даже в песочнице.
2. Подайте заявку в **App Store Small Business Program** (15% вместо 30%).
3. Приложение → Monetization → Subscriptions → группа «Selfvoy Pro» → продукты `selfvoy_pro_monthly` и `selfvoy_pro_yearly`: длительность, цена (Apple пересчитает для стран), локализация названия и описания, скриншот экрана оплаты для проверки, вводное предложение «7 дней бесплатно».
4. Users and Access → Integrations → **In-App Purchase** → сгенерировать ключ (.p8) → в секреты сервера.
5. App Information → **App Store Server Notifications**: Production URL `https://api.selfvoy.app/billing/apple`, Sandbox URL — тот же или отдельный, версия **2**. Проверка: App Store Server API «Request a Test Notification».
6. Users and Access → Sandbox → тестовые аккаунты (или войдите своим Apple Account в Настройки → Разработчик → Sandbox на iPhone).
7. Первую подписку отправляйте на проверку **вместе с версией приложения** (в карточке версии выбрать подписки).

Google Play Console

1. Настройки → **Платёжный профиль продавца** (без него нельзя создавать платные продукты).
2. Загрузите сборку **с подключённой Billing Library** хотя бы во внутренний тест — только после этого откроется создание подписок.
3. Монетизация → Подписки → `selfvoy_pro` → базовые планы `monthly` и `yearly` (автопродление, цены по странам, льготный период 7 дней, удержание аккаунта) → предложение `trial-7d` (новым клиентам).
4. Google Cloud: включить **Google Play Android Developer API**; создать сервисный аккаунт и пригласить его в Play Console (Пользователи и разрешения) с правами «Просмотр финансовых данных» и «Управление заказами и подписками».
5. Pub/Sub: тема `play-billing`, выдать `google-play-developer-notifications@system.gserviceaccount.com` роль Pub/Sub Publisher; push-подписка на `https://api.selfvoy.app/billing/google` с аутентификацией (OIDC, свой сервисный аккаунт, audience).
6. Монетизация → Настройка монетизации → **Real-time developer notifications**: имя темы → «Отправить тестовое уведомление».
7. Настройки → Тестирование лицензий: добавьте Google-аккаунты тестировщиков (покупки бесплатны, продления ускорены).

11. Экран оплаты: что обязательно

Элемент	Зачем
Название, срок и цена каждого варианта (из магазина), цена за период крупнее «экономии»	Apple 3.1.2, политика подписок Google — основная причина отказов
Для пробного периода: сколько длится и **сколько спишется потом**	Иначе отказ за вводящую в заблуждение подписку
Фраза об автопродлении и как отменить	Обязательное раскрытие условий
Ссылки «Условия использования» (EULA) и «Политика конфиденциальности» на экране оплаты	Apple проверяет их и в метаданных (поле EULA или описание)
Кнопка «Восстановить покупки»	Apple: обязательна; Google: желательна (resync)
«Управлять подпиской» в Настройках	Пользователь должен легко найти отмену
Free-версия остаётся полезной; экран Pro закрывается	Нельзя заставлять платить за базовый вход (Apple 3.1.2)
Перед удалением аккаунта — «подписка продолжит списываться, отмените её в магазине»	Apple 5.1.1(v), политика удаления аккаунта Google

12. Тестирование покупок

Среда	Как	Особенности
Xcode, StoreKit Configuration	Файл .storekit, запуск из Xcode	Без сети и без App Store Connect; можно ускорять время, вызывать ошибки и возвраты
Apple Sandbox / TestFlight	Тестовый аккаунт; сборки TestFlight всегда в песочнице	Продления ускорены (месяц ≈ 5 минут), до 12 продлений; уведомления — на Sandbox URL
Google: тестировщики лицензий	Внутренний тест + аккаунт из списка тестировщиков	Тестовые карты «всегда успешно / отказ»; месяц ≈ 5 минут; RTDN приходят как настоящие
Сервер	pytest с подписанными тестовыми данными и моками	Без сети; в CI вместе с `make ai-test`

Сценарии, которые надо пройти руками на обеих платформах

- Покупка месяца → Pro открылся сразу; перезапуск приложения — Pro на месте.
- Пробный период → после окончания списание и продление; отказ платёжного метода → grace → on_hold → истечение.
- Отмена автопродления → Pro до конца срока, потом Free; возврат денег → Pro пропал сразу (revoked).
- Смена месяц ↔ год; покупка на одном устройстве → вход на другом (в том числе iOS → Android) → Pro по аккаунту.
- Покупка под аккаунтом A → выход → вход B → «Восстановить покупки»: ответ `billing_conflict`, понятный текст.
- Обрыв сети сразу после оплаты → при следующем запуске покупка подхватывается (Transaction.updates / queryPurchasesAsync).
- Повторная доставка уведомления — ничего не ломает; удаление аккаунта с активной подпиской — предупреждение показано.

13. Техническая публикация: подпись, сборка, CI/CD

13.1. Версии

Один источник — файл `version.json` в корне (`{"version":"1.0.0"}`). Номер сборки = `GITHUB_RUN_NUMBER`: Android `versionCode`, iOS `CURRENT_PROJECT_VERSION`; версия для людей — `versionName` / `MARKETING_VERSION`. Каждая загрузка в магазин — новый номер сборки.

13.2. Android: подпись и AAB

Ключ загрузки

```
# один раз, локально; файл и пароли — в менеджер паролей и в секреты GitHub (две копии!)
keytool -genkeypair -v -keystore upload.jks -alias upload -keyalg RSA -keysize 2048 -validity 10000
base64 -w0 upload.jks   # → секрет ANDROID_UPLOAD_KEYSTORE
```

web/android/app/build.gradle (тот же смысл в синтаксисе Groovy)

```
android {
    signingConfigs {
        create("release") {
            storeFile = file(System.getenv("ANDROID_KEYSTORE_PATH") ?: "upload.jks")
            storePassword = System.getenv("ANDROID_KEYSTORE_PASSWORD")
            keyAlias = System.getenv("ANDROID_KEY_ALIAS")
            keyPassword = System.getenv("ANDROID_KEY_PASSWORD")
        }
    }
    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
            isMinifyEnabled = true            // R8; mapping.txt загружать в Play вместе с AAB
            proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
        }
    }
}
```

13.3. iOS: подпись и выгрузка

- Проще всего — автоматическая подпись Xcode с ключом App Store Connect API в CI (роль App Manager): `-allowProvisioningUpdates -authenticationKeyPath … -authenticationKeyID … -authenticationKeyIssuerID …`. Альтернатива — fastlane match (сертификаты в отдельном приватном репозитории).
- В `Info.plist`: `ITSAppUsesNonExemptEncryption = NO`; capability In-App Purchase и Push Notifications; `PrivacyInfo.xcprivacy`.
- Сейчас `build-ios.yml` ставит `latest-stable` Xcode — для релиза закрепите версию 26.x, чтобы сборка не менялась сама.

Сборка и загрузка iOS

```
xcodebuild archive -workspace web/ios/App/App.xcworkspace -scheme App -configuration Release \
  -destination "generic/platform=iOS" -archivePath build/Selfvoy.xcarchive \
  CURRENT_PROJECT_VERSION=$GITHUB_RUN_NUMBER MARKETING_VERSION=$VERSION \
  -allowProvisioningUpdates -authenticationKeyPath "$ASC_KEY_PATH" \
  -authenticationKeyID "$ASC_KEY_ID" -authenticationKeyIssuerID "$ASC_ISSUER_ID"

# ExportOptions.plist: method = app-store-connect, destination = upload, teamID = <TEAM_ID>
xcodebuild -exportArchive -archivePath build/Selfvoy.xcarchive \
  -exportOptionsPlist ExportOptions.plist -exportPath build/export \
  -allowProvisioningUpdates -authenticationKeyPath "$ASC_KEY_PATH" \
  -authenticationKeyID "$ASC_KEY_ID" -authenticationKeyIssuerID "$ASC_ISSUER_ID"
# destination=upload сразу отправляет сборку в App Store Connect → TestFlight
```

13.4. Workflow релиза

Схема workflow

```
# .github/workflows/release-mobile.yml — запуск по тегу v*.*.* (только из main)
on: { push: { tags: ["v*.*.*"] } }
permissions: { contents: read }
jobs:
  android:
    runs-on: ubuntu-latest
    environment: release            # секреты только здесь + обязательное ручное одобрение
    steps:
      - uses: actions/checkout@<SHA>
      - uses: actions/setup-node@<SHA>  # затем: cd web && npm ci && npm run build && npx cap sync android
      - run: echo "$ANDROID_UPLOAD_KEYSTORE" | base64 -d > web/android/app/upload.jks
      - run: cd web/android && ./gradlew bundleRelease -PversionCode=$GITHUB_RUN_NUMBER
      - uses: r0adkll/upload-google-play@<SHA>   # или fastlane supply
        with: { serviceAccountJsonPlainText: "${{ secrets.PLAY_SA_JSON }}", packageName: app.selfvoy,
                releaseFiles: "web/android/app/build/outputs/bundle/release/*.aab",
                track: internal,
                mappingFile: "web/android/app/build/outputs/mapping/release/mapping.txt" }
  ios:
    runs-on: macos-15               # с Xcode 26; закрепить версию явно
    environment: release
    steps:
      - uses: actions/checkout@<SHA>
      - run: cd web && npm ci && npm run build && npx cap sync ios
      - run: ./tools/ios-release.sh   # archive + exportArchive из 13.3
```

- Секреты: `ANDROID_UPLOAD_KEYSTORE`, `ANDROID_KEYSTORE_PASSWORD`, `ANDROID_KEY_ALIAS`, `ANDROID_KEY_PASSWORD`, `PLAY_SA_JSON`, `ASC_KEY_P8`, `ASC_KEY_ID`, `ASC_ISSUER_ID`, `AURA_SERVER_URL` — в GitHub Environment «release» с ручным одобрением.
- Действия GitHub закрепить по SHA (это уже есть в списке доработок отчёта). Выпуск в production — вручную в консолях после проверки сборки во внутреннем тесте/TestFlight.
- Сервисному аккаунту для загрузки в Play — отдельный, с правами только на выпуск в тестовые треки.

14. Прод-инфраструктура

Caddyfile (TLS-сертификат Caddy получает сам; для nginx — аналогичные location)

```
api.selfvoy.app {
    # WebSocket и HTTP основного сервера
    reverse_proxy /* server:9000
    # только вебхуки магазинов — в AI-сервис; остальные пути AI наружу не публикуются
    handle /billing/apple   { reverse_proxy ai:8000 }
    handle /billing/google  { reverse_proxy ai:8000 }
    handle /billing/rustore { reverse_proxy ai:8000 }
    header Strict-Transport-Security "max-age=31536000"
}
```

- TLS 1.2+ обязателен для уведомлений Apple; Pub/Sub push принимает только HTTPS с публичным сертификатом.
- За прокси все клиенты приходят с одного адреса — передавайте реальный IP заголовком и настройте лимиты сервера (см. SETUP).
- Резервные копии PostgreSQL ежедневно; таблицы подписок — критичные данные.
- Мониторинг: доля ошибок `billing.verify`, задержка вебхуков, число `billing_unavailable`; оповещение, если уведомления не приходят дольше суток.
- Логи без токенов покупок и JWS целиком — только ID транзакции и статус.
- Сверка раз в сутки: для подписок, у которых `expires_at` прошёл, а продления не было, запросить статус у магазина (страховка от потерянных уведомлений).

15. План работ

Этап	Что	Оценка
1	Миграция БД, роли, `.env.example`; модуль billing в AI-сервисе с тестами (Apple, Google, идемпотентность, конфликт)	4–5 дней
2	Протокол: billing.status/verify, событие, plan_limit; C++-сервер (вызов AI, право доступа, лимиты); PROTOCOL.md, чекеры, e2e	3–4 дня
3	Плагин Capacitor (StoreKit 2 + PBL 8), экран Pro, «Восстановить», «Управлять», предупреждение при удалении аккаунта	4–5 дней
4	Консоли: соглашения, продукты, ключи, Pub/Sub, вебхуки; прод-прокси	1–2 дня (+ ожидание одобрений)
5	Release-workflow, подпись, загрузка в TestFlight и внутренний тест	2 дня
6	Ручной прогон сценариев раздела 12 на iPhone и Android	2–3 дня
7	RuStore Pay SDK (если нужна аудитория в РФ)	3–4 дня

Итого ≈ 3–4 недели одному разработчику без RuStore. Нативные клиенты (`android/`, `ios/`) — отдельно, если они тоже идут в магазины.

Источники (проверены 26.10.2026)

• Play Billing Library deprecation: с 31.08.2026 новые приложения и обновления — PBL 8+, продление до 01.11.2026 (developer.android.com/google/play/billing/deprecation-faq).
• Play Developer API deprecations: subscriptions v1 → subscriptionsv2; subscriptionId в RTDN без замены (developer.android.com/google/play/billing/play-developer-apis-deprecations, обновлено 23.10.2026).
• Google Play: альтернативная оплата и внешние ссылки в США — отчётность и сборы с 01.10.2026 (support.google.com/googleplay/android-developer/answer/16497028, 16470497); новая модель сборов с 30.06.2026 (9to5google.com, 24.06.2026).

- Apple: App Store Server Notifications V2, App Store Server API, требования TLS 1.2 (developer.apple.com/documentation/appstoreservernotifications, appstoreserverapi).
- Apple: Small Business Program — 15% (developer.apple.com/app-store/small-business-program); условия в ЕС с 01.10.2026 (apple.com/newsroom, 18.08.2026).
- RuStore: BillingClient SDK прекращён 01.08.2026, Pay SDK (rustore.ru/help/sdk/payments/faq, rustore.ru/help/sdk/pay).
- RevenueCat: бесплатно до 2 500 USD MTR, далее 1% (revenuecat.com/pricing); плагин @revenuecat/purchases-capacitor (github.com/RevenueCat/purchases-capacitor).
- Статус платежей магазинов для пользователей из России — по общеизвестным заявлениям компаний 2022 года и публикациям 2026 года; перепроверьте перед запуском.